Based on Data Balancing and Model Improvement for Multi-Label Sentiment Classification Performance Enhancement study

1st Zijin Su*
University College London
UCL Dept of Physics
London, UK
1176564216@qq.com

1st Huanzhu Lv
Central South University
Dept of Computer Science and Technology
Changsha, China
lvhuanzhuchat@gmail.com

2nd Yuren Niu
Nanyang City Fifth Complete School
High school
Nanyang, China
zyb12668@163.com

3rd Yiming Liu
Wuhan Guanggu Future School
High school
Wuhan, China
lym20070118@outlook.com

***Abstract*—Multi-label sentiment classification identifies cases where a text expresses more than one emotion. Datasets such as GoEmotions have long-tail label distributions, which weaken performance for underrepresented emotions. This study therefore focuses on data construction, imbalance mitigation, and evaluation protocol design rather than claiming that a lightweight neural architecture outperforms Transformer models. We construct an imbalance-mitigated multi-label emotion dataset by integrating the original GoEmotions data, Sentiment140 tweets relabeled with a RoBERTa-base-GoEmotions classifier, and GPT-4 mini generated texts reviewed through an annotation and audit process. The resulting dataset contains 223,391 instances; its post-balancing label counts range from 6,129 to 20,107, so we describe it as more balanced rather than evenly balanced. On this dataset, we evaluate a CNN–BiLSTM–attention model with FastText embeddings and compare it with unbalanced ablations and a DistilRoBERTa baseline. We further add a duplicate-aware train-only augmentation rerun for DistilRoBERTa, in which external samples are added only to the training pool and no normalized-text overlap is found between training and held-out splits. Balancing improves the lightweight model relative to unbalanced training, while DistilRoBERTa obtains higher F1 and AUC scores. The study is therefore framed around dataset construction, quality control, leakage checks, and reproducible evaluation for long-tail multi-label emotion classification.**

***Index Terms*—Multi-label Sentiment Classification, Data Balancing, FastText, Bidirectional LSTM, Attention Mechanism**

Revision Note

This revision responds to Reviewer #2 by clarifying the data construction and split protocol, separating the augmentation-based dataset from the oversampled ablation setting, softening claims about balanced label distributions, adding annotation reliability and duplicate-text statistics, expanding error analysis, adding a stricter duplicate-aware train-only DistilRoBERTa rerun, and giving more reproducibility detail. Reviewer #1 reported that the previous concerns had been addressed and had no further comments.

I. Introduction

Sentiment analysis, a core task in natural language processing, systematically identifies and categorizes opinions expressed in text, typically classifying them as positive, negative, or neutral [1]. With the rapid growth of online communication, it has become indispensable for monitoring public opinion, interpreting customer feedback, and informing strategic decisions. In particular, online reviews can significantly influence public perception, and organizations increasingly rely on automated sentiment analysis to track brand reputation and consumer attitudes [2].

Despite its broad application, sentiment analysis faces persistent challenges. One of the most prominent is data imbalance, where certain emotion categories are severely underrepresented, leading to weaker performance for minority classes [3]. Another difficulty lies in detecting implicit sentiment, such as sarcasm, irony, or wordplay, where the literal meaning of words diverges from the intended message [4]. These issues are amplified in multi-label settings, where a single text can express multiple overlapping emotions. Traditional solutions—such as oversampling, undersampling, and cost-sensitive learning—are not universally effective: oversampling can lead to overfitting, undersampling risks losing important context, and cost-sensitive methods require careful parameter tuning to avoid bias.

To address these limitations, this study focuses on dataset construction, imbalance mitigation, and an explicit evaluation protocol for fine-grained multi-label emotion classification. We augment GoEmotions with two additional sources: tweets from Sentiment140 re-labeled into 28 GoEmotions categories using the SamLowe/roberta-base-go-emotions classifier [5], and 20k GPT-4 mini generated samples designed to increase coverage of long-tail emotions. Both sources undergo filtering, annotation review, and sample-based audit checks. The resulting augmented dataset is not uniform; it reduces the severity of the long-tail distribution while retaining a realistic multi-label structure. We then evaluate a lightweight CNN–BiLSTM–attention architecture as a reproducible baseline and include DistilRoBERTa comparisons under both the released construction-then-split dataset and a stricter train-only augmentation protocol. The paper treats the data balancing pipeline, quality-control evidence, leakage checks, and released artifacts as the main focus, with the lightweight model serving as an interpretable and efficient baseline rather than a replacement for Transformer models.

II. Related work

Previous studies on sentiment analysis have examined diverse datasets, algorithms, and classification strategies. Zuo [6] analyzed 7.7 million Steam game reviews from multiple sources, comparing Gaussian Naïve Bayes and Decision Tree classifiers with TF–IDF features. The Decision Tree achieved 75% accuracy, benefiting from feature selection through Information Gain or Gini Index. Likewise, Devika et al [7] compared three sentiment analysis approaches—machine learning, rule-based, and lexical-based—detailing seven machine learning classifiers, and found that machine learning generally yielded higher precision, F1-score, and accuracy.

Bashynska et al. [8] applied a pre-trained BERT model to the GoEmotions dataset, achieving a macro-average F1 score of 0.5070 and demonstrating BERT's effectiveness in multi-label emotion classification. However, the study did not address dataset imbalance, leading to weaker performance on minority classes. Building on these findings, our work targets improved classification performance while explicitly addressing class imbalance, with the goal of enhancing results for underrepresented emotions and improving applicability in real-world settings.

III. Methodology

A. Dataset preparation

GoEmotions [9] (Kaggle) contains Reddit comments annotated with 28 emotion labels (12 positive, 11 negative, 4 ambiguous, 1 neutral). After removing samples with `example_very_unclear`, 207,814 instances remain. To mitigate long-tail imbalance, we augmented the pool with (i) Sentiment140 [10] tweets automatically labeled into 28 GoEmotions categories using SamLowe/roberta-base-go-emotions [5] (labels stored in the `model_labels` field and mapped to the corresponding GoEmotions names; empty sets mapped to *neutral*), and (ii) 20k GPT-4 mini generated texts targeting rare labels. All labels follow the official GoEmotions 28-label taxonomy; no additional semantic remapping was applied beyond string normalization.

Quality control combined model-based filtering and human review. Let $p_k(x)$ denote the predicted probability for label k on sample x. We keep a sample only if $\max_k p_k(x) \geq 0.7$; labels are assigned by $p_k(x) \geq 0.5$ with a top-3 cap and empty sets discarded. GPT samples follow the same verification before being reviewed by five annotators with majority-vote resolution. The Sentiment140 weak-label pool contains 700,000 tweets, with label cardinality 576,423 (1 label), 115,347 (2 labels), 8,029 (3 labels), and 201 (≥ 4 labels). The GPT-4 mini set contains 20,000 texts; the annotation log reports unanimous agreement (5/5) for all 20,000 entries. We audited 200 randomly sampled items per added source, yielding 200/200 matches for Sentiment140 and 200/200 for GPT-4 mini (Table I). These audits are sample-based and do not preclude residual noise.

Dataset construction proceeded in two stages. First, we formed an augmented candidate pool from the filtered GoEmotions data, the Sentiment140 weak-label pool, and the GPT-4 mini generated samples. Second, we applied a greedy balancing procedure with per-label caps: a maximum of 9,000 samples per label, with higher caps for *neutral* (20,000) and *annoyance* (15,000). Multi-label samples update all label counts simultaneously, so exact caps are approximate. The final augmented balanced dataset contains 223,391 instances and 196,401 unique texts. Its per-label counts range from 6,129 (*nervousness*) to 20,107 (*neutral*), so the dataset mitigates but does not eliminate the long-tail imbalance. We report the label cardinality distribution in Table II and the per-label counts before and after balancing in Table VII. This augmentation-based balanced dataset is distinct from the oversampled ablation setting discussed later in Table III.

TABLE I: Audit summary for added sources.

Source	Pool Size	Audit Size	Match Rate
Sentiment140 auto-labels	700000	200	1.00
GPT-4 mini (manual labels)	20000	200	1.00

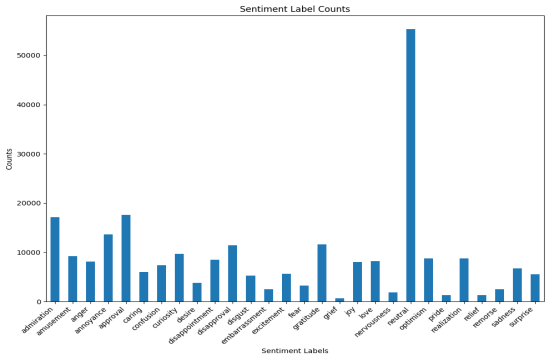


Fig. 1: Label distribution of the original GoEmotions dataset after removing `example_very_unclear`.

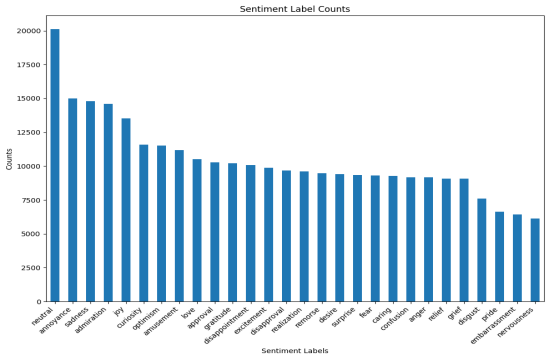


Fig. 2: Label distribution after augmentation-based balancing (Sentiment140 + GPT-4 mini).

After assembling the dataset, we applied a consistent text preprocessing pipeline. We removed @mentions and URLs, normalized whitespace, and filtered non-textual artifacts. After constructing the augmented balanced dataset, we created stratified train/validation/test splits using *MultilabelStratifiedShuffleSplit* (random_state=42): 178,786 / 22,274 / 22,331

TABLE II: Label cardinality distribution (number of labels per sample).

Labels per Sample	Original	Balanced
1	171820 (82.68%)	160464 (71.83%)
2	31187 (15.01%)	57299 (25.65%)
3	4218 (2.03%)	5162 (2.31%)
≥4	589 (0.28%)	466 (0.21%)

samples (80/10/10). A word-level Tokenizer was fit on the training set only, and all splits were transformed using the same vocabulary with a maximum sequence length of 30 tokens. Emotion labels were converted into 28-dimensional multi-hot vectors using MultiLabelBinarizer. Figure 3 presents the 50 most frequent words in the balanced training set.

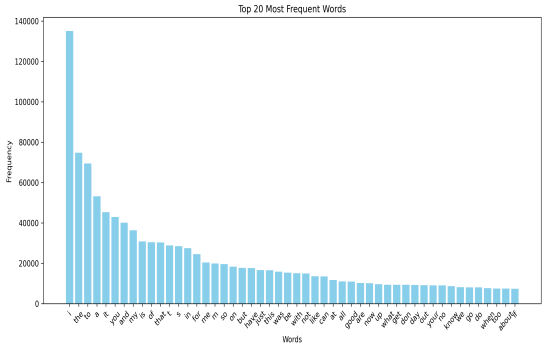


Fig. 3: Top-50 most frequent words in the balanced training data.

B. Leakage Control and Split Protocol

Two split protocols are now reported and kept separate. The primary released augmented dataset uses a construction-then-split protocol: external-source augmentation and greedy label balancing are performed before the stratified 80/10/10 split. For this protocol, model-fitting leakage control is maintained because the tokenizer is fit only on the training split, validation data are used for threshold selection and model comparison, and the held-out test split is used only for final reporting. Because this protocol evaluates performance on the augmented distribution, we also ran a stricter duplicate-aware train-only augmentation protocol for the DistilRoBERTa check: the original GoEmotions data were first deduplicated and stratified into 46,213/5,730/5,734 train/validation/test rows; Sentiment140 and GPT-4 mini samples were then added only to the training pool after removing normalized-text overlaps with the validation and test splits. Greedy balancing produced 189,945 training rows, and the final exact normalized-text overlap was 0 for train–validation and 0 for train–test. These strict-protocol results are reported separately in Table VI and are not mixed with the construction-then-split ablation results.

C. Data processing

We compute label co-occurrence statistics on the training split and visualize cosine similarity (Fig. 4). The embedding layer was initialized with a pre-trained FastText matrix to map tokens to 300-dimensional word vectors.

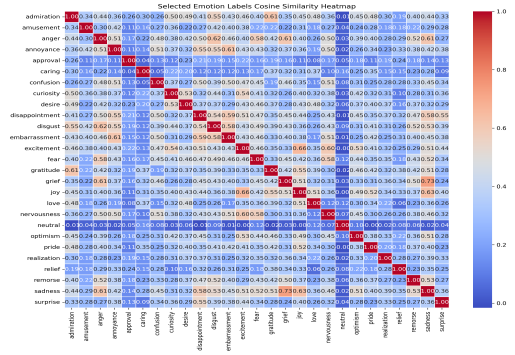


Fig. 4: This figure shows the cosine similarity heat map of emotion labels.

After preprocessing, the unbalanced GoEmotions split and the imbalance-mitigated split were used for the corresponding ablation and final-model experiments. In brief, the model pipeline processes embedded word vectors through convolution, recurrent, and, when applied, attention layers, followed by pooling operations and a sigmoid output layer to produce independent probability scores for each emotion label. The overall workflow is illustrated in Fig. 5.

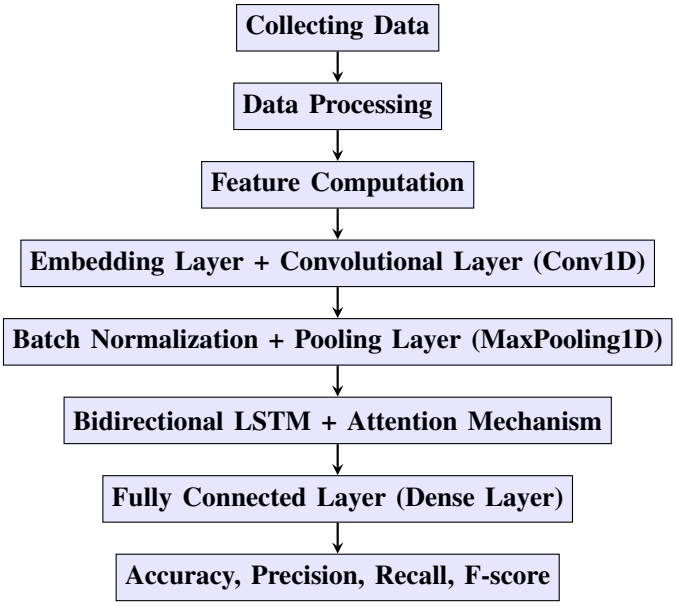


Fig. 5: The step by step workflow of the process of the whole experiment

D. Model

The model starts with a 300-dimensional FastText embedding layer. We freeze the embedding matrix during training to reduce overfitting and to keep the parameter-efficient setting. Freezing the embedding matrix yields 0.33M trainable parameters out of 21.5M total (Table X). A 1D convolution layer

(64 filters, kernel size 5, stride 1) is applied along the token sequence to capture local n-gram patterns, followed by batch normalization and max-pooling to downsample the sequence representation. A bidirectional LSTM (128 units per direction) then models longer-range dependencies. When enabled, an attention module reweights time steps to emphasize emotion-relevant tokens; otherwise, we use temporal average pooling. Finally, a dense layer (128 units) with dropout (0.5) is followed by a sigmoid output layer producing independent probabilities for 28 emotion labels. The full layer-by-layer specification is provided in the Appendix (Table X).

E. Training process

All experiments used the same optimization setup: Adam optimizer (lr $= 1 \times 10^{-3}, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-7}$), batch size 256, and 34 training epochs (no early stopping for the ablation curves). Automatic mixed precision (AMP) was enabled to improve training efficiency. For threshold tuning, we performed per-label grid search on the validation set to maximize label-wise F1, using thresholds from 0.05 to 0.95 with a step size of 0.05. The resulting threshold vector τ was applied once to the held-out test set for evaluation (no test information was used in tuning).

To examine the contribution of attention and data balancing, we conducted ablation experiments under three configurations: (i) unbalanced data with attention, (ii) unbalanced data without attention, and (iii) balanced data with attention (final).

Figure 6 shows the unbalanced+attention setting. Training loss decreases steadily, while validation loss starts increasing earlier than training loss, indicating mild overfitting. This divergence is likely driven by the severe label imbalance and weak-label noise in the unbalanced setting, where the model increasingly fits spurious patterns that reduce training loss but do not transfer to validation. In addition, attention can amplify dataset-specific cues, making the model more sensitive to rare-label artifacts and thus accelerating validation loss growth. In this setup, attention reweights token positions to emphasize informative cues for multi-label prediction.

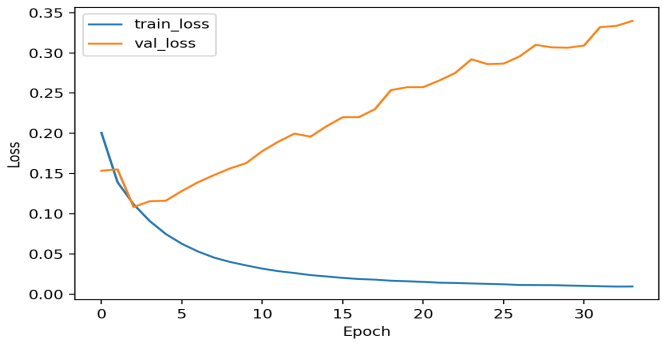


Fig. 6: Training and validation loss curves for the unbalanced GoEmotions dataset with attention.

Figure 7 shows the unbalanced setting without attention. Validation loss reaches its minimum earlier and then increases, indicating stronger overfitting when attention is removed. In this case, temporal average pooling after the BiLSTM is used instead of attention.

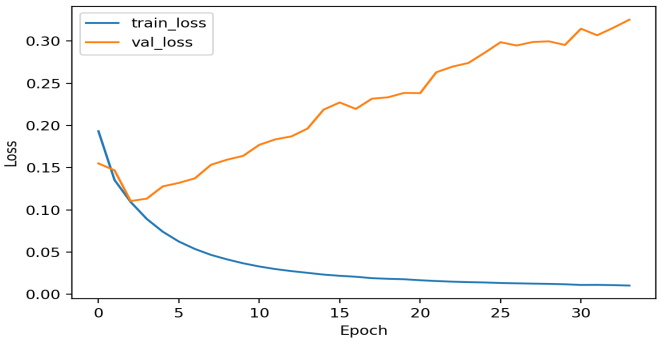


Fig. 7: Training and validation loss curves for the unbalanced GoEmotions dataset without attention.

Finally, we train the augmentation-based balanced dataset with attention as the lightweight final model and use it as the primary reference for subsequent evaluations. Figure 8 reports its training/validation loss curves; both losses decrease and stabilize over epochs, indicating stable optimization under the imbalance-mitigated setting.

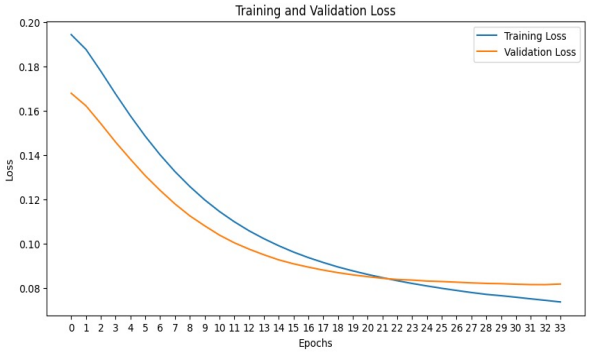


Fig. 8: Training and validation loss curves over 34 epochs for the balanced GoEmotions training set with attention (final model).

F. Evaluation Metrics

For multi-label classification, we report the following metrics. Unless otherwise stated, predicted labels are obtained by thresholding sigmoid outputs at 0.5.

- **Subset Accuracy (Exact Match):** the proportion of samples whose predicted label set exactly matches the ground-truth label set.
- **Jaccard Index:** the average, over samples, of the intersection-over-union between predicted and true label sets.
- **Hamming Loss:** the fraction of incorrect label assignments averaged over all sample–label pairs.

- **Micro Precision/Recall/F1:** computed by aggregating true positives, false positives, and false negatives over all labels.
- **Macro Precision/Recall/F1:** the unweighted mean of per-label precision/recall/F1.
- **Macro ROC-AUC:** the mean ROC-AUC computed independently for each label.

In multi-label settings, the commonly reported accuracy in Table VIII is the macro average of one-vs-rest label accuracies, which can be high because most labels are negative for most samples. For the balanced+attention model, macro per-label accuracy is ~ 0.966 at a 0.5 threshold and ~ 0.946 after threshold optimization, while subset accuracy remains around 0.31. We therefore emphasize Jaccard and F1 as primary indicators.

G. Implementation

We implemented the CNN–BiLSTM experiments in TensorFlow 2.x (Keras), using NumPy and Pandas for data handling, scikit-learn for evaluation utilities (e.g., MultiLabelBinarizer and metric computation), Gensim to load pre-trained FastText embeddings, NLTK for text preprocessing, and `iterative-stratification` for multi-label splitting. Experiments were run on an NVIDIA Tesla V100 GPU (16 GB) with an Intel Xeon CPU and 64 GB RAM. For reproducibility, we fix the stratified split with `random_state=42`, use an 80/10/10 train/validation/test split, and tune per-label thresholds over the validation set using a grid from 0.05 to 0.95 in steps of 0.05. The released scripts include `scripts/data_balancing_pipeline.py`, `scripts/cnn_bilstm_training.py`, `zenodo_upload/transformer_baseline_train.py`, and the new `scripts/reviewer2_revision_analysis.py`, which generates the revision statistics under `analysis/reviewer2/`.

IV. Analysis and Discussion

A. Baseline vs Ablation study

To evaluate multi-label classification, we report subset (exact-match) accuracy, Jaccard index, micro-/macro-averaged F1, and macro-averaged ROC-AUC under a fixed decision threshold of 0.5 for consistent comparison across ablations. We use subset accuracy (rather than label-wise accuracy) because the latter can be inflated by the large number of negative labels in multi-label settings. Table III summarizes the ablation results under this fixed-threshold protocol. For clarity, the word "balanced" refers to two related but separate settings in this paper. The main dataset is the augmentation-based balanced dataset constructed from GoEmotions, Sentiment140, and GPT-4 mini samples. In contrast, the balanced setting in Table III is an oversampled GoEmotions training split used only to isolate the effect of balancing within the ablation comparison. We do not treat this oversampled ablation as the same object as the augmentation-based released dataset.

Table III shows that attention improves performance on the unbalanced data, and balancing further boosts all metrics (e.g., Macro-F1 from 0.164 to 0.201). Our final model uses the balanced+attention setting. We also report per-label F1 (threshold=0.5) for a fine-grained comparison across emotions, see following figures.

TABLE III: Ablation study results on the GoEmotions test set (threshold = 0.5).

Setting	Subset Acc	Jaccard	Micro-F1	Macro-F1	Macro AUC
Unbalanced + Attn	0.305	0.334	0.431	0.164	0.821
Unbalanced - Attn	0.237	0.263	0.372	0.149	0.820
Balanced + Attn	0.311	0.341	0.438	0.201	0.829

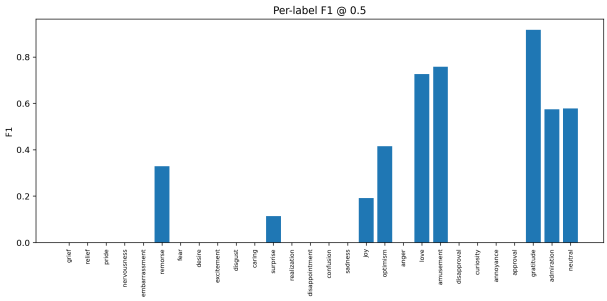


Fig. 9: Per-label F1-scores (0.5 threshold) for the Unbalanced + Attention configuration.

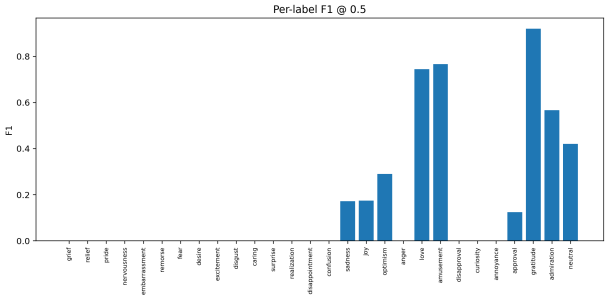


Fig. 10: Per-label F1-scores (0.5 threshold) for the Unbalanced - Attention configuration.

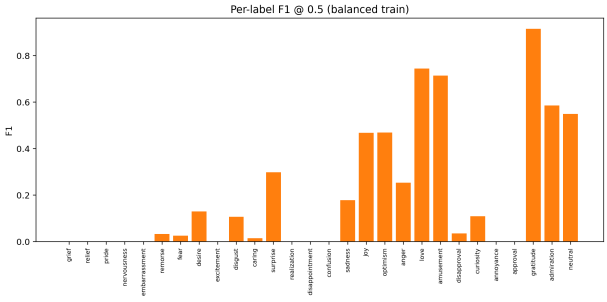


Fig. 11: Per-label F1-scores (0.5 threshold) for the Balanced + Attention configuration.

We next compare our final balanced+attention model against transformer baselines.

B. Comparison with Transformer Baselines

We fine-tuned DistilRoBERTa-base on the augmented balanced dataset using the same stratified construction-then-split protocol (train/validation/test = 178,786/22,274/22,331). Table IV reports results at threshold 0.5, Table V summarizes the gap relative to our lightweight model, and Fig. 12 visualizes the comparison. Table VI additionally reports a stricter duplicate-aware train-only augmentation rerun for DistilRoBERTa.

TABLE IV: DistilRoBERTa baseline on the balanced dataset (threshold = 0.5).

Model	Subset Acc	Jaccard	Micro-F1	Macro-F1	Macro AUC
DistilRoBERTa (balanced)	0.627	0.710	0.776	0.773	0.980

TABLE V: Gap analysis between DistilRoBERTa and our model (threshold = 0.5).

Metric	Our Model	DistilRoBERTa	Delta
Subset Accuracy	0.311	0.627	+0.316
Jaccard	0.341	0.710	+0.369
Micro-F1	0.438	0.776	+0.338
Macro-F1	0.201	0.773	+0.572
Macro AUC	0.829	0.980	+0.151

TABLE VI: DistilRoBERTa under duplicate-aware train-only augmentation.

Threshold	Subset Acc	Jaccard	Micro-F1	Macro-F1	Macro AUC
0.5	0.139	0.463	0.557	0.474	0.864
Validation-tuned	0.090	0.462	0.579	0.508	0.864

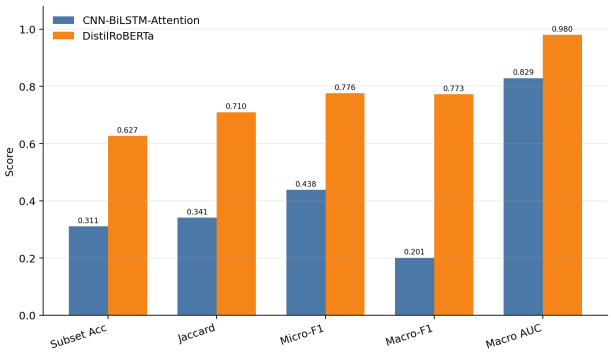


Fig. 12: Bar chart comparison of DistilRoBERTa and our model (threshold = 0.5).

DistilRoBERTa obtains higher Micro-F1, Macro-F1, and Macro-AUC than the lightweight model on the construction-then-split augmented dataset (Table V), which is expected given its pretraining and stronger contextual representations. Under validation-tuned thresholds, the same augmented-dataset rerun reaches Micro-F1 0.777, Macro-F1 0.777, and Macro-AUC 0.980. The stricter train-only augmentation rerun gives a lower but more conservative estimate: validation-tuned Micro-F1 0.579, Macro-F1 0.508, and Macro-AUC 0.864 on held-out original GoEmotions test data, with no exact normalized-text overlap between training and held-out splits. The two DistilRoBERTa settings answer different questions: the construction-then-split result measures performance on the augmented distribution, while the train-only result tests generalization back to the original GoEmotions distribution under stronger leakage controls. We therefore use the Transformer results to contextualize the lightweight model and treat the data construction and evaluation protocol as the main contribution.

C. Single-emotion performance evaluation

As an auxiliary analysis, we performed per-label threshold optimization: for each label, we selected a decision threshold on the validation set to maximize label-wise F1, producing a threshold vector τ. This vector was applied once to the held-out test set for evaluation (no test information was used in tuning). Since this corresponds to a different operating point, these results are reported separately in Table VIII.(see Appendix)

Table VIII summarizes per-label performance under the optimized thresholds. Note that per-label accuracy is computed in a one-vs-rest manner and can be inflated by true negatives; we therefore emphasize F1 and AUC for interpretation. Overall, the model achieves consistently high ROC-AUC across labels, while performance varies in F1 for semantically subtle and context-dependent emotions (e.g., approval and disappointment), which often overlap with related categories and exhibit higher annotation ambiguity.

To illustrate model behavior, Table IX lists the top predicted labels and probabilities for representative sentences. The model assigns a dominant label when the emotion is explicit (e.g., sadness in "I feel bad for those boys") and can capture co-occurring emotions in multi-faceted sentences (e.g., "good news ... bad news ..."). More colloquial or implicit expressions lead to more dispersed probabilities across labels, highlighting the challenge of ambiguity and context in multi-label emotion recognition.

D. Evaluation and Discussion

Overall, the experiments show that imbalance mitigation improves the lightweight model relative to unbalanced training, although the size of the gain is modest compared with the gap to DistilRoBERTa. In the ablation comparison, the balanced+attention setting improves Macro-F1 from 0.164 to 0.201 and Jaccard from 0.334 to 0.341 over the unbalanced+attention configuration (Table III). These results indicate that balancing helps under the fixed-threshold ablation setup, but they should be read together with the Transformer reruns: DistilRoBERTa achieves higher Micro-F1, Macro-F1, and Macro-AUC than the lightweight CNN–BiLSTM–attention model, and the train-only protocol shows how performance changes under a stricter leakage-control setting.

In the construction-then-split DistilRoBERTa optimized-threshold output, the lowest-F1 labels are *approval* (F1=0.592,

support=1029), *disgust* (F1=0.615, support=759), *disappointment* (F1=0.658, support=1009), and *disapproval* (F1=0.661, support=969). Under the stricter train-only protocol, the weakest labels are *pride* (F1=0.136, support=113), *grief* (F1=0.209, support=56), *relief* (F1=0.268, support=109), and *nervousness* (F1=0.284, support=156). The pattern suggests two different error sources. First, semantically close labels such as *approval*, *disapproval*, and *disappointment* remain difficult even in the augmented split. Second, rare labels in the original GoEmotions validation and test sets become especially difficult when the held-out data are not augmented. Low optimal thresholds for several weak labels further suggest that threshold calibration is compensating for uncertain decision boundaries.

Data quality also remains a limitation. The GPT-4 mini annotation log records unanimous five-annotator agreement for all 20,000 generated items, and the 200-sample audits for both Sentiment140 and GPT-4 mini report 200/200 matches. These checks provide limited reliability evidence, but they do not prove that the labels are noise-free because the audit is sample-based and the weak-label source depends on another model. The duplicate-text analysis further shows 223,391 rows but only 196,401 unique texts, with 26,990 duplicate rows (12.08%) and a maximum duplicate count of 224. Many high-frequency duplicates are generated grief-style templates, which may improve rare-label coverage but also risk teaching template-specific language patterns. This is another reason the paper avoids claiming a fully balanced or fully noise-controlled dataset.

Future work should extend the stricter train-only augmentation protocol to all lightweight and Transformer models, increase the size of independent double-annotated audits for weak and generated sources, and test noise-aware objectives for semantically overlapping labels under the same documented split and threshold-tuning protocol.

V. Conclusion

This paper investigates multi-label emotion classification under severe class imbalance. We propose an augmentation-based data construction pipeline that combines GoEmotions, Sentiment140 weak labels, and GPT-4 mini generated examples, then applies greedy label balancing to mitigate the long-tail distribution. The resulting dataset is more balanced but not evenly balanced, and the revision explicitly reports remaining label-count variation, duplicate-text statistics, and audit limitations.

Experiments show that balancing improves a lightweight CNN–BiLSTM–attention baseline relative to unbalanced training, while DistilRoBERTa performs better on F1 and AUC metrics. The added duplicate-aware train-only rerun further shows that stricter leakage control lowers absolute scores but gives a more conservative estimate on held-out original GoEmotions data. Accordingly, the study does not claim model superiority over Transformers; it documents a dataset balancing, quality-control, leakage-checking, and reproducibility workflow for long-tail multi-label emotion classification. Future work will extend the strict train-only protocol to all models, enlarge independent annotation audits, and explore stronger noise-robust modeling.

VI. Acknowledgement

We express our sincere gratitude to Professor Björn Schuller for his valuable feedback, patience, and continuous support throughout this research. Special thanks are also extended to teaching assistant Weihao Tang for his insightful guidance during project development. Zijin Su and Huanzhu Lv contributed equally to this work and are recognized as co-first authors.

VII. Author Contributions

Zijin Su and Huanzhu Lv contributed equally to this work and are recognized as co-first authors. Zijin Su led the conceptualization, data curation, formal analysis, investigation, and project administration, and contributed to both the original draft and its revision. Huanzhu Lv supported the conceptualization, methodology, resources, and validation, and contributed to data analysis and manuscript editing. Yuren Niu contributed to visualization and overall review of the manuscript. Yiming Liu provided resources and participated in the revision of the manuscript. All authors have read and approved the final version of the manuscript.

VIII. Funding

This research received no external funding.

IX. Conflict of interest

The authors declare that they have no conflict of interest.

X. Data Availability

The balanced datasets (final_balanced_df_output.csv and balanced_emotion_dataset.csv), the Sentiment140 auto-labelled pool (sentiment140_with_go_emotions_labels.csv), audit logs (sentiment140_audit.csv, gpt4mini_annotations.csv, and gpt4mini_audit.csv), label count tables, ablation summaries, and Transformer baseline outputs supporting the findings of this study are provided. Additional supporting files, including label_distribution_summary.csv, duplicate_text_summary.csv, audit_reliability_summary.csv, and low_f1_error_summary.csv, are also included. The stricter train-only augmentation split and rerun were produced using scripts/build_train_only_augmented_splits.py and scripts/train_distilroberta_from_splits.py; the retained revision artefacts include split_summary.json, per-label metric CSV files, optimised thresholds, and the run script required to reproduce the Transformer reruns. The core dataset and figures are openly available on Zenodo at https://doi.org/10.5281/zenodo.15837871, https://doi.org/10.5281/zenodo.16890154, and https://doi.org/10.5281/zenodo.18366640 (records 15837871, 16890154, and 18366640).

References

[1] Bing Liu. *Sentiment analysis and opinion mining*. Springer Nature, 2022.

[2] Achmad Bayhaqy, Sfenrianto Sfenrianto, Kaman Nainggolan, and Emil R Kaburuan. Sentiment analysis about e-commerce from tweets using decision tree, k-nearest neighbor, and naïve bayes. In *2018 international conference on orange technologies (ICOT)*, pages 1–6. IEEE, 2018.

[3] Nilaa Raghunathan and Kandasamy Saravanakumar. Challenges and issues in sentiment analysis: A comprehensive survey. *IEEE Access*, 11:69626–69642, 2023.

[4] Da Li, Rafal Rzepka, Michal Ptaszynski, and Kenji Araki. Hemos: A novel deep learning-based fine-grained humor detecting method for sentiment analysis of social media. *Information Processing & Management*, 57(6): 102290, 2020.

[5] Sam Lowe. roberta-base-go_emotions (revision 58b6c5b). Hugging Face, 2024. URL https://huggingface.co/SamLowe/roberta-base-go_emotions.

[6] Zhen Zuo. Sentiment analysis of steam review datasets using naive bayes and decision tree classifier. 2018.

[7] M Devi Devika, C[a] Sunitha, and Amal Ganesh. Sentiment analysis: a comparative study on different approaches. *Procedia Computer Science*, 87:44–49, 2016.

[8] Iryna Bashynska, Mykhailo Sarafanov, and Olga Manikaeva. Research and development of a modern deep learning model for emotional analysis management of text data. *Applied Sciences*, 14(5):1952, 2024.

[9] Dorottya Demszky, Dana Movshovitz-Attias, Jeongwoo Ko, Alan Cowen, Gaurav Nemade, and Sujith Ravi. Goemotions: A dataset of fine-grained emotions. *arXiv preprint arXiv:2005.00547*, 2020.

[10] Isnaini Nurul Khasanah. Sentiment classification using fasttext embedding and deep learning model. *Procedia Computer Science*, 189:343–350, 2021.

XI. Appendix

For reproducibility, we provide the full per-label threshold optimization results, raw evaluation outputs, additional plots, source scripts, and revision statistics in the repository and Zenodo records listed in the Data Availability section. The construction-then-split seed is 42, the train/validation/test ratio is 80/10/10, and validation-only threshold tuning uses thresholds from 0.05 to 0.95 with a step size of 0.05. The added train-only rerun also uses seed 42, DistilRoBERTa-base, three epochs, maximum sequence length 64, learning rate 2×10^{-5}, and validation-only threshold tuning. Its split summary records 46,213/5,730/5,734 original GoEmotions train/validation/test rows, 189,945 balanced training rows after adding external training-only samples, and zero normalized-text overlap between training and held-out splits. The label-count table and detailed model specification are shown on the next page.

TABLE VII: Label counts before and after balancing.

Emotion	Original	Balanced	Emotion	Original	Balanced
admiration	17131	14603	amusement	9245	11188
anger	8084	9176	annoyance	13618	15001
approval	17620	10285	caring	5999	9290
confusion	7359	9185	curiosity	9692	11580
desire	3817	9404	disappointment	8469	10085
disapproval	11424	9690	disgust	5301	7586
embarrassment	2476	6422	excitement	5629	9868
fear	3197	9314	gratitude	11625	10225
grief	673	9065	joy	7983	13518
love	8191	10510	nervousness	1810	6129
optimism	8715	11515	pride	1302	6621
realization	8785	9607	relief	1289	9087
remorse	2525	9476	sadness	6758	14789
surprise	5514	9337	neutral	55298	20107

TABLE VIII: Per-label performance under validation-tuned thresholds (accuracy is one-vs-rest per label).

Emotion	Accuracy	Precision	Recall	F1-Score	AUC	Support	Threshold
Admiration	0.9596	0.7079	0.6628	0.6846	0.9521	1460	0.35
Amusement	0.9843	0.8389	0.8480	0.8435	0.9814	1119	0.25
Anger	0.9696	0.8398	0.5585	0.5964	0.9454	918	0.35
Annoyance	0.9376	0.5327	0.6216	0.5737	0.9343	1500	0.35
Approval	0.9507	0.4855	0.4511	0.4216	0.8969	1029	0.20
Caring	0.9723	0.7011	0.6549	0.6771	0.9575	929	0.25
Confusion	0.9731	0.7656	0.5476	0.6385	0.9494	919	0.35
Curiosity	0.9555	0.5866	0.5580	0.5719	0.9137	1158	0.20
Desire	0.9824	0.8111	0.7423	0.7752	0.9723	940	0.40
Disappointment	0.9553	0.5411	0.4660	0.5008	0.9116	1009	0.20
Disapproval	0.9541	0.4786	0.5496	0.5117	0.9328	969	0.35
Disgust	0.9646	0.4933	0.5674	0.5278	0.9413	759	0.30
Embarrassment	0.9851	0.8581	0.5814	0.6932	0.9422	642	0.40
Excitement	0.9692	0.6580	0.6361	0.6468	0.9502	987	0.25
Fear	0.9842	0.8086	0.7978	0.8031	0.9751	931	0.55
Gratitude	0.9904	0.9074	0.8737	0.8903	0.9841	1023	0.40
Grief	0.9918	0.9336	0.8480	0.8888	0.9909	896	0.25
Joy	0.9691	0.7698	0.6833	0.7239	0.9645	1352	0.30
Love	0.9858	0.8350	0.8649	0.8497	0.9796	1051	0.35
Nervousness	0.9859	0.8548	0.5906	0.6986	0.9466	613	0.40
Neutral	0.9155	0.5135	0.5466	0.5295	0.9045	2011	0.30
Optimism	0.9776	0.8361	0.7190	0.7731	0.9642	1152	0.30
Pride	0.9942	0.9924	0.8065	0.8898	0.9667	662	0.20
Realization	0.9647	0.5734	0.4711	0.5172	0.9143	961	0.20

TABLE IX: Examples of top predicted emotion labels and probabilities (sigmoid outputs).

Sentence	Top-1	Top-2	Top-3	Top-4
wants to fall in love	desire: 0.98	love: 0.86	-	-
My mom liked her mothers day gift SUCCESS	love: 0.82	excitement: 0.63	-	-
yes hand of fatima or hamsa i am tryna start a collection if you find one great if you don t it s cool just have FUN	joy: 0.72	optimism: 0.55	amusement: 0.42	-
I feel bad for those boys	sadness: 0.93	-	-	-
People done be ignorant	neutral: 0.54	disappointment: 0.34	-	-
good news TONS of new people signing up for my virtual cocktail party thursday bad news my site just went down	surprise: 0.73	sadness: 0.68	joy: 0.65	disappointment: 0.55

TABLE X: Model: Text_Sentiment_Classification_Model

Layer (type)	Output Shape	Param #	Connected to
input_layer (InputLayer)	(None, 30)	0	-
embedding_layer (Embedding)	(None, 30, 300)	21,210,600	input_layer[0][0]
conv_layer (Conv1D)	(None, 26, 64)	96,064	embedding_layer[0][0]
batch_normalization (BatchNormalization)	(None, 26, 64)	256	conv_layer[0][0]
max_pooling1d (MaxPooling1D)	(None, 13, 64)	0	batch_normalization[0][0]
bidirectional (Bidirectional)	(None, 13, 256)	197,632	max_pooling1d[0][0]
dense (Dense)	(None, 13, 1)	257	bidirectional[0][0]
lambda (Lambda)	(None, 13)	0	dense[0][0]
activation (Activation)	(None, 13)	0	lambda[0][0]
lambda_1 (Lambda)	(None, 13, 1)	0	activation[0][0]
multiply (Multiply)	(None, 13, 256)	0	bidirectional[0][0], lambda_1[0][0]
lambda_2 (Lambda)	(None, 256)	0	multiply[0][0]
dense_layer (Dense)	(None, 128)	32,896	lambda_2[0][0]
dropout (Dropout)	(None, 128)	0	dense_layer[0][0]
output_layer (Dense)	(None, 28)	3,612	dropout[0][0]
Total params	21,541,317	(82.17 MB)	
Trainable params	330,589	(1.26 MB)	
Non-trainable params	21,210,728	(80.91 MB)	